55259e91-54a3-4635-83e0-c88484403278

<prev_user>

<request>

You are working on an issue in the 'hilo-mpc/hilo-mpc' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/hilo-mpc/hilo-mpc (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-15T07:08:09.141Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>Write documentation on observers</issue_title>
<issue_description>We need to expand the of the Observer moduls in the documentation (sphynx). At the moment there is only a bit of the Kalman Filter. but all the rest is missing

1) Look at the control module to have an idea on what kind of information the Observer module should have
2) Complete the observer module documentation</issue_description>

Comments on the Issue (you are @copilot in this section)

<comments>
</comments>

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

Perfect! Let me create a final summary of the work completed:

Summary

I have successfully completed the task of expanding the Observer module documentation in the HILO-MPC repository. Here's what was accomplished:

Changes Made

File Modified: `docs/docsource/modules/estimators.rst` (expanded from 54 to 356 lines, a 659% increase)

Documentation Added for All 5 Observer Types:

1. **Moving Horizon Estimator (MHE)**
   - Overview and problem formulation
   - Setup requirements (horizon, stage/arrival costs)
   - Initialization examples
   - Cost function configuration
 parison with standard KF

4. **Unscented Kalman Filter (UKF)**
   - Overview of unscented transformation
   - Sigma point approach
   - UKF-specific tuning parameters (alpha, beta, kappa)
   - Advantages over EKF

5. **Particle Filter (PF)**
   - Overview of Sequential Monte Carlo method
   - Use cases (nonlinear, non-Gaussian, multimodal)
   - Particle count configuration
   - Variants and options (roughening, prior editing)

Additional Features:

- **18 practical code examples** showing initialization and usage
- **"Choosing the Right Observer" decision guide** with:
  - System characteristics criteria
  - Computational cost comparison
  - Estimation requirements mapping
  - Comprehensive comparison table with clarifying notes
- **Cross-reference** to existing MHE chemical reaction example
- **Consistent structure** following the Control module documentation style

Quality Assurance:

✅ RST syntax validated
✅ All section headers properly formatted
✅ Class and method references correct
✅ Code review feedback addressed
✅ Comprehensive coverage of all observer types
✅ Professional documentation quality

The Observer module documentation now provides users with complete information to understand, choose, and implement the appropriate observer for their state estimation needs, matching the quality and comprehensiveness of the Control module documentation.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'hilo-mpc/hilo-mpc' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/hilo-mpc/hilo-mpc (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-15T07:55:38.367Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Expand Observer module documentation with comprehensive content for all estimator types
</pr_title>

<pr_description>
Overview

This PR addresses issue #[issue_number] by significantly expanding the Observer module documentation to provide comprehensive coverage of all state estimation methods available in HILO-MPC.

Problem

The Observer module documentation was minimal (~50 lines) with only basic information about the Kalman Filter. Users lacked guidance on:
- How to set up and use the different observer types
- When to choose one observer over another
- Practical examples and code snippets
- Configuration options and parameters

Solution

Expanded `docs/docsource/modules/estimators.rst` from 54 to 356 lines (659% increase) following the structure and style of the Control module documentation.

Documentation Added

1. **Moving Horizon Estimator (MHE)**
- Problem formulation and use cases
- Setup requirements (horizon length, cost functions)
- Stage and arrival cost configuration
- Measurement handling and estimation workflow
- Support for non-uniform sampling intervals and multi-rate measurements
- Cross-reference to the MHE chemical reaction example

````
#### 2. **Kalman Filter (KF)**
- Overview for linear systems with Gaussian noise
- Initialization and setup instructions
- Noise covariance matrix configuration (Q and R)
- Prediction and update workflow

#### 3. **Extended Kalman Filter (EKF)**
- Overview for nonlinear systems
- Linearization approach explanation
- Setup instructions with same interface as KF
- When to use EKF vs KF

#### 4. **Unscented Kalman Filter (UKF)**
- Unscented transformation approach
- Sigma point method explanation
- UKF-specific tuning parameters (alpha, beta, kappa)
- Advantages over EKF for highly nonlinear systems

#### 5. **Particle Filter (PF)**
- Sequential Monte Carlo method overview
- Use cases: nonlinear, non-Gaussian, multimodal distributions
- Particle count configuration
- Variants and options (roughening, prior editing)

#### 6. **Observer Selection Guide**
- "Choosing the Right Observer" decision criteria
- System characteristics (linearity, noise type)
- Computational cost comparison
- Estimation requirements (constraints, parameters, real-time)
- Comprehensive comparison table with clarifying notes

## Key Features

- **18 practical code examples** demonstrating initialization
and usage patterns
- **Consistent structure** matching the Control module
documentation style
- **Proper cross-references** to API documentation and examples
- **Decision support** to help users select the appropriate
observer
- **Professional formatting** with validated RST syntax

## Example

Before (only Kalman Filter had minimal info):
```python
````
````

````
from hilo_mpc import KF
kf = KF(model, plot_backend='bokeh')
```

After (comprehensive setup for all observers):
```python
from hilo_mpc import MHE

# Initialize MHE with a model
mhe = MHE(model, plot_backend='bokeh')

# Set horizon length
mhe.horizon = 10

# Configure stage cost for measurements
mhe.quad_stage_cost.add_measurements(names=['y1', 'y2'],
weights=[10, 10])

# Configure arrival cost
mhe.quad_arrival_cost.add_states(names=['x1', 'x2'], weights=[1,
1])

# Setup and estimate
mhe.setup(options={'integration_method': 'collocation'})
x_estimated = mhe.estimate(x_arrival=x0)
```

## Testing

- ✅ RST syntax validated
- ✅ All section headers properly formatted
- ✅ Class and method references correct
- ✅ Code examples follow repository conventions
- ✅ Structure consistent with Control module

## Impact

This enhancement significantly improves the usability of HILO-
MPC's Observer module by providing users with:
1. Complete coverage of all 5 observer types
2. Clear guidance on observer selection
3. Practical, copy-paste ready code examples
4. Professional documentation matching project standards
````
````

Fixes hilo-mpc/hilo-mpc#53
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3405047855</comment_id>
<author>@brunomorampc</author>
@copilot I added some math in the MHE section (but it is not exactly correct).
1) Fix teh MHE formulation using this paper as reference As reference you can use this paper
2203.13671v1.pdf
2) Add math to the other methods (e.g. KF and the rest)
3) make the link between math and equations
</comment_new>

</pr_comments>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can

```
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```